

KARTHIKEYAN KUMARAGURUPARAN

Los Angeles, CA | 213-292-0970 | kk08430@usc.edu
karthikeyank1206.github.io | linkedin.com/in/karthikeyan-k5 | github.com/KarthikeyanK1206

SUMMARY

Software engineer specializing in backend and distributed systems. Year-long software engineering internship building production multi-tenant microservices, durable Temporal workflows, and serverless AWS onboarding; now completing an MS in Computer Science at USC (May 2027). Systems projects focus on failure behavior: quorum-replicated storage, crash-safe LLM-agent execution, and a Linux energy profiler published on GitHub.

EDUCATION

University of Southern California

Master of Science in Computer Science — GPA: 3.53/4.00

Relevant Coursework: Distributed Systems, Algorithms & Data Structures, Database Management

Aug 2025 – May 2027

Los Angeles, CA

Kumaraguru College of Technology

Bachelor of Engineering in Computer Science and Engineering — GPA: 9.23/10.0

Nov 2020 – May 2024

Coimbatore, India

TECHNICAL SKILLS

Languages: Go, Python, C++, Java, TypeScript, JavaScript, SQL, Bash

Backend & APIs: Node.js, Express.js, REST APIs, gRPC, Protocol Buffers, Temporal, Microservices

Cloud & DevOps: AWS (Lambda, API Gateway, IAM, KMS), Docker, Jenkins, Ansible, Nginx, Linux, Git

Databases & Storage: PostgreSQL, MySQL, MongoDB, Redis, RocksDB, SQLite

Distributed Systems: Quorum replication, write-ahead logging (WAL), crash recovery, idempotency, fault injection

PROFESSIONAL EXPERIENCE

Clar Technologies — Client: Skuchain

Software Engineer Intern · Node.js, Temporal, AWS, Docker, Ansible, MongoDB

Jul 2023 – Jul 2024

Coimbatore, India

- Built Node.js/Express microservices for a multi-tenant supply-chain-financing platform, running long multi-step financing operations as durable Temporal workflows that survive restarts and partial failures.
- Built serverless onboarding and key-management flows on AWS Lambda (API Gateway, IAM, KMS) serving 500+ users; documented 8+ partner REST APIs for external integrators.
- Cut environment provisioning from ~60 minutes to ~5 with Ansible playbooks; containerized services with Docker and standardized Jenkins delivery pipelines across a distributed team.
- Operated the platform's Nginx reverse proxy for 12 months without a recorded outage.

Samsung PRISM — Samsung R&D Institute India

Project Intern — raplscope: Linux System Energy Profiler · Go, Linux

Mar 2023 – Sep 2023

Remote

- Created raplscope, a zero-dependency Go CLI measuring whole-system energy and power from Linux RAPL counters via powercap sysfs, sampling down to a validated 10 ms floor; since rebuilt end to end and published at github.com/KarthikeyanK1206/raplscope.
- Engineered wraparound-safe accounting: wrap-corrected deltas (counters wrap roughly every 45 min at ~100 W), boundary snapshots so sub-interval commands measure correctly, monotonic-clock power math, and package-only summation to avoid double counting.
- Added /usr/bin/time-style command wrapping — inherited streams, SIGINT/SIGTERM forwarding, exit-code propagation (128+N on signals) — with JSON/CSV export feeding a bundled HTML energy dashboard.
- Made hardware-bound logic testable without root or RAPL hardware by injecting the sysfs root; 12 table-driven tests run against a fake counter tree.

PROJECTS

NimbusVault — Adaptive Distributed Object Store

C++17, gRPC, Protocol Buffers, RocksDB · Group Project

- Focused on the metadata plane: a RocksDB-backed write-ahead log replicated from a fixed-leader coordinator across a 3-node metadata cluster with quorum commits — deliberately scoped below full Raft (no election or membership changes).
- Designed adaptive replication: heartbeat-driven access windows classify chunks hot/warm/cold with asymmetric hysteresis (3 windows to promote, 5 to demote), scaling chunks between 2 and 5 replicas — cold data holds 2 replicas vs. a static 3, a one-third cold-storage reduction by construction.
- Built two-stage replica reconfiguration: a quorum-committed START keeps the old set readable as fallback while new copies are made and verified; a COMMIT then publishes the stable set, so reads stay valid mid-move.
- Authored 69 unit tests in 10 GoogleTest files, 8 integration scenarios including node-kill faults, and a Wing–Gong linearizability checker for client histories.

RAEF — Fault-Tolerant Execution Runtime for LLM Agents

Python, SQLite, LangChain, OpenRouter · Group Project

- Designed a log-before-send transaction layer: after a mid-tool-call crash, a restart distinguishes replayable work from ambiguous writes, and dedup-capable targets recognize retries by deterministic SHA-256-derived execution IDs.
- Implemented crash recovery over a SQLite WAL with synchronous=FULL durability: a coordinator classifies interrupted executions by lifecycle state — intent-logged work replays; dispatched work waits, verifies, or hands off rather than guessing.
- Modeled capability-aware recovery (dedup, check APIs, visible state, opaque targets) and exercised the runtime across 9 crash-injection phases with a 25-test pytest suite.

PUBLICATION

S. Barath Sanjay, B. Harish, K. Karthikeyan, et al., "IoT Based Smart IV Drip Stand," *IEEE Conference*, Oct. 2021.